

AgroSentinela

IA como Fertilizante Digital — um novo agronegócio do amanhã

Global Solution 2026.1 | FIAP | Fase 7 | Grupo 14

Vitorio Stevanatto Compri Paciulo

RM56789 | vitorioscp@gmail.com

QUERO CONCORRER

Plataforma de baixo custo que une sensor ESP32, dados climáticos reais e duas Inteligências Artificiais para prever risco hídrico, otimizar a irrigação por cultura e alertar o pequeno produtor por voz.

1. Introdução

A pergunta central desta Global Solution é: “Como a IA e as tecnologias digitais podem transformar a nova economia espacial e gerar impacto positivo na Terra?” O **AgroSentinela** responde levando os dados do espaço e do céu até quem efetivamente planta. Sensoriamento — clima, umidade do solo e NDVI orbital (Embrapa/INPE) — vira decisão prática de irrigação na mão do pequeno produtor.

O problema é concreto: o pequeno produtor irriga “**no escuro**”, ora desperdiçando água, energia e dinheiro, ora descobrindo tarde que a lavoura entrou em estresse hídrico. A solução combina hardware barato (ESP32 simulado no Wokwi), uma fonte de dados pública e gratuita (Open-Meteo) e **duas IAs** que, juntas, dizem ao produtor **se, quando e quanto irrigar** cada cultura — comunicando por voz, interface acessível a quem tem pouca familiaridade com telas. É a metáfora do “**drone barato de grande impacto**”: tecnologia simples e acessível com efeito enorme no campo.

2. Desenvolvimento

A solução integra todas as disciplinas da fase num fluxo de ponta a ponta, do sensor no campo até o alerta falado que chega ao produtor.

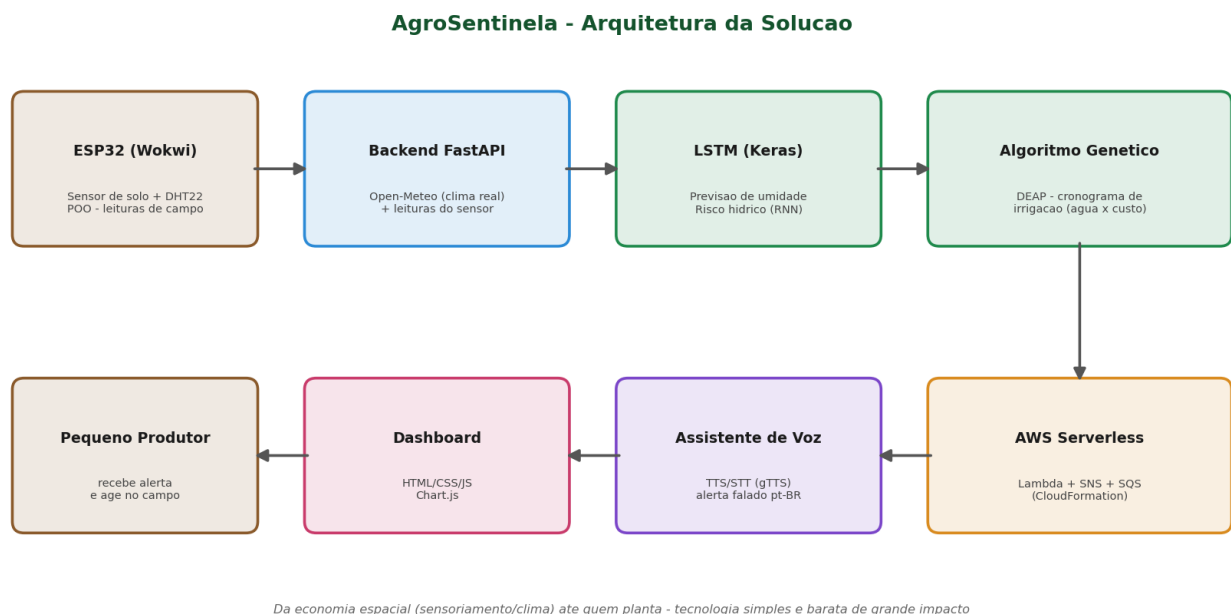


Figura 1 — Arquitetura integrada do AgroSentinela.

2.1. Coleta de dados climáticos reais (Open-Meteo)

O módulo `coleta_dados.py` consome a API histórica da Open-Meteo (gratuita, sem chave) para **Ribeirão Preto/SP: 17.496 registros horários** (2 anos) de temperatura, umidade do ar, precipitação, evapotranspiração e umidade/temperatura do solo. O backend também consulta a **condição climática atual em tempo real** (forecast API), exibida no painel.

```
def coletar(latitude, longitude, data_inicio, data_fim, timeout=60):
    parametros = {
        "latitude": latitude, "longitude": longitude,
        "start_date": data_inicio, "end_date": data_fim,
        "hourly": " ".join(VARIAVEIS_HORARIAS), "timezone": "America/Sao_Paulo",
    }
    resposta = requests.get(URL_OPEN_METEO, params=parametros, timeout=timeout)
    resposta.raise_for_status()
```

```
df = pd.DataFrame(resposta.json()["hourly"])
df["time"] = pd.to_datetime(df["time"])
return df.sort_values("time").reset_index(drop=True)
```

2.2. IA 1 — Previsão de risco hídrico com Rede Neural LSTM (Cap. 2)

Uma rede recorrente **LSTM** (Keras/TensorFlow) aprende a dinâmica temporal da umidade do solo. Empilha duas camadas LSTM (64 e 32) com dropout e prevê a umidade da próxima hora a partir de uma janela de 24 horas. Atingiu **MAE** $\approx 0,010 \text{ m}^3/\text{m}^3$ no teste — frente à média de $0,258 \text{ m}^3/\text{m}^3$.

```
def construir_modelo(n):
    md = Sequential([Input(shape=(JANELA, n)), LSTM(64, return_sequences=True),
                    Dropout(0.2), LSTM(32), Dropout(0.2),
                    Dense(16, activation="relu"), Dense(1)])
    md.compile(optimizer="adam", loss="mse", metrics=["mae"])
    return md
```

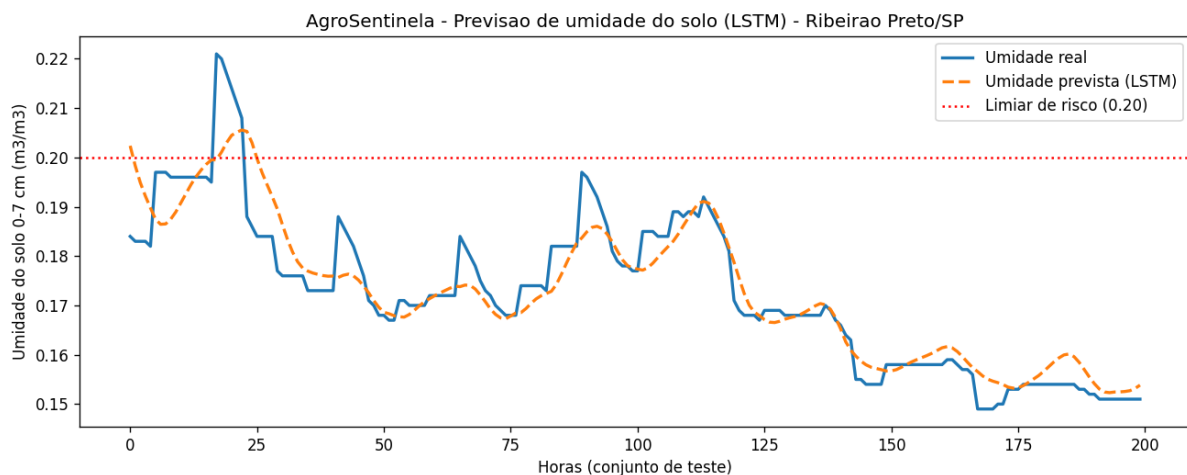


Figura 2 - Validação do modelo LSTM no teste: umidade real x prevista no conjunto de teste (avaliação da rede, não é a tela do app).

2.3. IA 2 — Otimização da irrigação com Algoritmo Genético (Cap. 4)

Um **algoritmo genético** (DEAP) busca o cronograma de irrigação de 24h que mantém a umidade no alvo da cultura gastando a **menor quantidade de água**. Cada indivíduo é um vetor de 24 genes (mm/hora); a aptidão soma o custo da água a uma penalidade pelo déficit de umidade.

```
def avaliar(ind):
    ir = np.array(ind); u = simular_umidade(ir)
    custo = float(np.sum(ir))*CUSTO_POR_MM
    deficit = np.maximum(0.0, UMIDADE_ALVO-u)
    return (custo + float(np.sum(deficit))*PESO_DEFICIT,)
```

2.4. Previsão que REAGE ao sensor + irrigação por CULTURA

Diferencial central: a previsão **parte da leitura real do sensor** (rebaseada sobre a curva da LSTM) e o cronograma é recalculado ao vivo para o alvo da cultura. Assim, o mesmo solo gera recomendações diferentes conforme o que foi plantado, e os gráficos do painel mudam conforme o dado do ESP32.

```
def previsao_ao_vivo(u):
    d = u - SERIE_LSTM[0]
    return [max(0.03, round(v+d, 4)) for v in SERIE_LSTM]

def cronograma_ao_vivo(prev, alvo):
    irr, com, ef = [], [], 0.0
    for base in prev:
        ef *= (1-DECAIMENTO); us = base+ef; i = 0.0
        if us < alvo: i = min(MAX_MM_HORA, (alvo-us)/GANHO_POR_MM); ef += i*GANHO_POR_MM
        irr.append(round(i,2)); com.append(round(base+ef,4))
    return irr, com
```

Cada uma das 10 culturas mais plantadas do Brasil tem limiar e alvo próprios. A tabela mostra a água recomendada para o mesmo cenário climático — note o contraste de água entre as culturas; os gráficos reais aparecem no painel (Figura 4):

Cultura	Umid. crítica	Umid. alvo	Água 24h
Arroz	0,35	0,42	≈ 40 mm
Café	0,22	0,28	≈ 18 mm
Feijão	0,22	0,28	≈ 18 mm
Milho	0,20	0,27	≈ 16 mm
Soja / Trigo	0,20	0,26	≈ 15 mm
Algodão	0,19	0,25	≈ 13 mm
Cana-de-açúcar	0,18	0,24	≈ 11 mm
Sorgo	0,16	0,22	≈ 8 mm
Mandioca	0,15	0,20	≈ 5 mm

2.5. Nó de campo ESP32 — 4 sensores, POO (Cap. 8)

Um único ESP32 simula **4 campos** (um potenciômetro por cultura), programado de forma orientada a objetos: as classes `SensorSolo` e `NoDeCampo` encapsulam a leitura e o envio das medições em JSON, via HTTP, para o backend.

```
class NoDeCampo {
private: SensorSolo solo; String id; String cultura;
public:
    NoDeCampo(uint8_t pino, String idCampo, String cult) : solo(pino), id(idCampo), cultura(cult) {}
    void iniciar() { solo.iniciar(); }
    void publicar(float temperatura, float umidadeAr) {
        if (isnan(temperatura)) temperatura = 25.0;
        if (isnan(umidadeAr)) umidadeAr = 50.0;
        String json = "{";
        json += "\"id_fazenda\": \"" + id + "\", ";
        json += "\"umidade_solo\": " + String(solo.lerUmidade(), 3) + ", ";
        json += "\"temperatura\": " + String(temperatura, 1) + ", ";
        json += "\"umidade_ar\": " + String(umidadeAr, 1) + ", ";
        json += "\"cultura\": \"" + cultura + "\"";
        json += "}";
        if (WiFi.status() != WL_CONNECTED) return;
        HTTPClient http; String url = String(URL_BACKEND);
        if (url.startsWith("https")) {
            WiFiClientSecure cli; cli.setInsecure();
            http.begin(cli, url); http.addHeader("Content-Type", "application/json");
            http.addHeader("ngrok-skip-browser-warning", "true");
            int cod = http.POST(json); Serial.println("[ " + id + " ] HTTP " + String(cod) + " | " + json);
        } else {
            http.begin(url); http.addHeader("Content-Type", "application/json");
            int cod = http.POST(json); Serial.println("[ " + id + " ] HTTP " + String(cod) + " | " + json);
        }
        http.end();
    }
};
```

2.6. Backend FastAPI multi-ESP32 e alertas serverless (Caps. 1, 5 e 6)

O FastAPI consolida o sistema: aceita **vários ESP32 ao mesmo tempo** (cada um um campo), serve o painel e dispara o alerta quando há risco. Os alertas são processados de forma assíncrona e serverless: um template **CloudFormation** declara fila SQS, Lambda, tópico SNS, DynamoDB e papel IAM de menor privilégio.

```
def handler(event, context):
    processadas = 0; alertas = 0
```

```

for registro in event.get("Records", []):
    leitura = json.loads(registro["body"]); processadas += 1
    if dynamo:
        dynamo.Table(TABELA).put_item(Item={"id_fazenda": leitura["id_fazenda"],
            "timestamp": dt.datetime.utcnow().isoformat(), "umidade_solo": str(leitura["umidade_solo"])}))
    if avaliar(leitura):
        alertas += 1
        msg = ("Risco hidrico na fazenda " + leitura["id_fazenda"] + ": umidade " +
            str(leitura["umidade_solo"]) + " m3/m3 (limiar " + str(LIMIAR) + ").")
        if sns and TOPICO: sns.publish(TopicArn=TOPICO, Subject="AgroSentinela - Alerta", Message=msg)
        print("[SNS] " + msg)
    return {"processadas": processadas, "alertas": alertas}

```

2.7. Assistente de voz e painel (Caps. 3 e 7)

O alerta é comunicado por **voz** em português (gTTS), e o painel — pensado para o agricultor — mostra cada campo com semáforo verde/vermelho, barra de umidade, conselho em linguagem simples (“regar X litros por m²”), os dois gráficos das IAs e botão de voz.

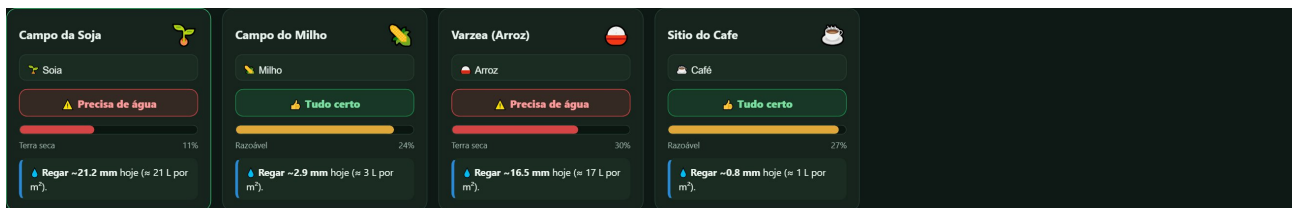


Figura 3 - Painel real do produtor: cada cartão e um sensor ESP32, com semáforo, umidade e conselho de rega.



Figura 4 - Detalhe de um campo no painel real: KPIs e os dois gráficos das IAs (LSTM e Algoritmo Genético) ao vivo.

2.9. Decisões do projeto

Algumas escolhas que guiaram o desenvolvimento: (1) **Região de Ribeirão Preto/SP**, polo agrícola com histórico climático rico; (2) **Open-Meteo** como fonte de dados, por ser real, gratuita e sem chave; (3) **LSTM** para a previsão, por ser rede recorrente com memória temporal, ideal para séries de umidade; (4) a **previsão e rebaseada na leitura do sensor**, para reagir ao campo real e não só ao histórico; (5) o **algoritmo genético** para a otimização, pois minimizar água sem deixar a planta em risco não tem fórmula fechada e tem muitas combinações; (6) **limiares por cultura** (10 culturas), pois cada lavoura tem necessidade hídrica diferente; (7) um **ESP32 com 4 sensores**, que demonstra vários campos numa única simulação; (8) **AWS simulada via CloudFormation**, mantendo a arquitetura serverless real com custo zero; e (9) uma **interface simples e por voz**, porque o público-alvo é o pequeno produtor.

2.8. Segurança — Red Team vs Blue Team (Cap. 9)

Ameaça (Red Team)	Defesa (Blue Team)
Injeção de leituras falsas	API key por dispositivo, validação de faixa, rate limiting, HMAC
Spoofing do ESP32	Identidade única (AWS IoT/mTLS), rotação de credenciais
DoS na API	API Gateway com throttling, WAF, fila SQS (assíncrono)
Interceptação (MITM)	TLS/HTTPS obrigatório fim a fim
Escalada de privilégio	IAM de menor privilégio (ver template.yaml)
Vazamento de dados (LGPD)	Criptografia em repouso/trânsito, minimização de dados

3. Resultados Esperados

Indicador	Resultado
Dados climáticos reais coletados	17.496 horas (2 anos, Ribeirão Preto/SP)
Erro da previsão de umidade (LSTM)	MAE $\approx$ 0,010 m ³ /m ³ (teste)
Culturas suportadas	10 (soja, milho, cana, feijão, arroz, algodão, trigo, café, mandioca, sorgo)
Água otimizada por cultura	de $\approx$ 5 mm (mandioca) a $\approx$ 40 mm (arroz) por 24h
Economia vs. irrigação no escuro	superior a 75% no volume de água
Campos simultâneos (multi-ESP32)	ilimitado; demo com 4 sensores num ESP32
Infra serverless	7 recursos CloudFormation (SQS, Lambda, SNS, DynamoDB, IAM)

O AgroSentinela entrega agricultura de precisão ao pequeno produtor com custo próximo de zero: economiza água, reduz a conta de energia do bombeamento e respeita a necessidade real de cada cultura — uso sustentável e inteligente dos recursos hídricos.

4. Conclusões

O AgroSentinela demonstra, na prática, a tese da Global Solution: a economia espacial — sensoriamento, dados climáticos e índices orbitais — gera impacto positivo na Terra quando esses dados chegam, traduzidos em ação, a quem produz alimento. Integrando coleta de dados reais, uma rede neural LSTM de previsão, um algoritmo genético de otimização por cultura, um nó ESP32 com quatro sensores orientado a objetos, alertas serverless e um assistente de voz, o projeto cobre todas as disciplinas da fase num sistema coeso, funcional e que roda de verdade.

A principal lição do desenvolvimento foi o equilíbrio entre **simplicidade e impacto**: cada decisão priorizou baixo custo e acessibilidade sem abrir mão de rigor — dados reais, validação do modelo, segurança por design e adaptação a cada lavoura. Ver a previsão reagir à leitura do sensor e o plano de rega mudar entre arroz e mandioca tornou tangível o valor de unir as duas IAs. Como evolução, pretende-se incorporar o NDVI orbital diretamente ao modelo, calibrar os limiares com agrônomos em campo e implantar a infraestrutura AWS em ambiente real com hardware físico.

Links da entrega

Repositorio do projeto (GitHub): <https://github.com/viscp1011/AgroSentinela>

Video de demonstracao (YouTube, nao listado): <https://youtu.be/g0mi0LDirxM>

QUERO CONCORRER — AgroSentinela | Vitorio Stevanatto Compri Paciulo |
FIAP Global Solution 2026.1